

Universidad Pontificia Comillas

ICAI

CUADERNO DE TRABAJO

Aprendizaje por refuerzo

Lydia Ruiz Martínez, Pablo Tuñón Laguna, Alberto Velasco Rodríguez

**APLICACIÓN DE CONCEPTOS GENERALES
DE RL**



Curso 2025-2026

Índice

1. Navegación con tile coding	2
1.1. Representación y generalización	2
1.2. Aproximación lineal e interpretación del gradiente	2
1.3. Recompensa en <code>step()</code> : diseño, escala y riesgos	2
1.4. Funciones <code>get_q_values</code> y <code>update</code>	3
1.5. Exploración de ϵ	3
1.6. Parámetros	3
1.7. Comparativa SARSA vs Q-learning	3
2. Operación en almacén	6

1. Navegación con tile coding

Esta sección documenta el diseño, las decisiones y los resultados de la práctica de navegación con aproximación lineal y *tile coding*. El objetivo es aprender una política que desplace al agente en un mapa de 10 m por 10 m hasta una zona objetivo evitando colisiones, usando acciones discretas de desplazamiento cardinal con paso de 0.25 m. Se parte de una observación cruda $(x, y, \text{collision}, \text{target})$ y se construye una representación dispersa mediante mosaicos solapados.

1.1. Representación y generalización

El estado continuo no se presta a una tabla de valores, así que se emplea *tile coding* con varias rejillas solapadas. Cada rejilla activa exactamente un índice, por lo que el vector de entrada $\mathbf{x}(s)$ tiene exactamente n_{tilings} unos y el resto ceros. La normalización por anchura y altura de tile desacopla la escala del mapa de la discretización. Se aplican desplazamientos distintos por rejilla con paso relativo (3, 1) en los ejes para romper alineamientos regulares y evitar patrones repetitivos que degradan la generalización. Con $n_{\text{tiles_width}} = 10$, $n_{\text{tiles_height}} = 10$ y $n_{\text{tilings}} = 8$ se obtiene un compromiso útil: resolución suficiente en pasillos y solapamiento para transferir experiencia entre posiciones cercanas.

Los pesos se almacenan como una matriz $\mathbf{weights} \in \mathbb{R}^{\text{num_actions} \times \text{feature_size}}$. Esta forma evita ambigüedades de indexado y permite sumar de manera vectorizada en las columnas correspondientes a los índices activos.

1.2. Aproximación lineal e interpretación del gradiente

Se aproxima $Q(s, a)$ con un modelo lineal por acción,

$$Q(s, a) \approx \mathbf{w}_a^\top \mathbf{x}(s),$$

donde $\mathbf{x}(s)$ es binaria y dispersa. La derivada de Q respecto a $\mathbf{w}_a$ es exactamente $\mathbf{x}(s)$, así que el gradiente está implícito al actualizar únicamente las coordenadas activas. En cada transición se modifican n_{tilings} pesos de la fila asociada a la acción ejecutada. Esto aconseja escalar la tasa de aprendizaje con el número de rejillas; como regla práctica, $\alpha \approx 0,1/n_{\text{tilings}}$ ofrece magnitudes razonables.

1.3. Recompensa en step(): diseño, escala y riesgos

El cálculo de la recompensa se integra en `step()` con tres casos:

$$r = \begin{cases} +100, & \text{si el agente entra en la zona objetivo,} \\ -100, & \text{si hay colisión,} \\ -c - k(|x - c_x| + |y - c_y|), & \text{en otro caso,} \end{cases}$$

donde (c_x, c_y) es el centro de la zona objetivo. El término L1 genera una pendiente constante hacia la meta y evita efectos indeseados de la norma L2 (gradientes pequeños lejos del objetivo y posibles ganancias positivas al cuadrar). La escala de los

extremos y del shaping es crítica: si los terminales son demasiado grandes dominan la señal y aparecen trayectorias absurdas; si c y k son demasiado pequeños, falta guía intermedia. Una horquilla inicial razonable es $c \in [0,05, 0,1]$ y $k \in [0,005, 0,02]$. Para atascos o colisiones sistemáticas se puede reducir $|100|$, subir c para penalizar pasos inútiles o ajustar k para hacer más pronunciada la pendiente.

Cuando una acción intenta cruzar una pared, conviene no mover la posición y terminar el episodio por colisión. La comprobación de llegada se hace sobre la posición efectiva tras validar colisiones.

1.4. Funciones `get_q_values` y `update`

`get_q_values` suma los pesos en los índices activos para cada acción. De forma vectorizada,

$$\text{q_values} = \text{weights[:, idx].sum(axis=1)}.$$

En SARSA (semi-gradiente):

$$\delta = \begin{cases} r - Q(s, a), & \text{si termina,} \\ r + \gamma Q(s', a') - Q(s, a), & \text{en otro caso,} \end{cases} \quad \mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha \delta \mathbf{x}(s).$$

En Q-learning se sustituye $Q(s', a')$ por $\max_{a'} Q(s', a')$. En ambos casos se actualizan solo las coordenadas activas.

1.5. Exploración de ϵ

La política de comportamiento es ϵ -greedy con decaimiento exponencial. Tres parámetros la gobiernan: fracción de episodios a partir de la cual empieza el decaimiento, razón de decaimiento y valor mínimo. Iniciar muy pronto o decaer agresivo cierra la exploración; iniciar muy tarde ralentiza la consolidación. En los ensayos fue equilibrado empezar entre el 50–70 % de episodios, con razón en $[0,95, 0,99]$ y mínimo en $[0,02, 0,05]$, de modo que al final la política sea casi determinista (como exige la evaluación).

1.6. Parámetros

Se fijaron $n_{\text{tiles_width}} = 10$, $n_{\text{tiles_height}} = 10$, $n_{\text{tilings}} = 8$, zona objetivo $(2,5, 8, 1,0, 2,0)$, $\gamma = 0,99$ y 10,000 episodios. Los hiperparámetros más sensibles fueron α y el calendario de ϵ : un decaimiento excesivo degradó por cierre prematuro; posponer demasiado el inicio enlentece la convergencia. El refactor incluyó centralizar código, corregir la forma de `weights` a matriz bidimensional y exponer parámetros antes embebidos.

1.7. Comparativa SARSA vs Q-learning

Se entrenaron ambos agentes con los mismos mosaicos, $\gamma = 0,99$, 10,000 episodios, y un calendario de ϵ con decaimiento tardío. En este entorno determinista,

Q-learning suele consolidar trayectorias más rápidas; SARSA tiende a políticas algo más conservadoras si hay estocasticidad. Las figuras siguientes muestran las métricas de cada agente:

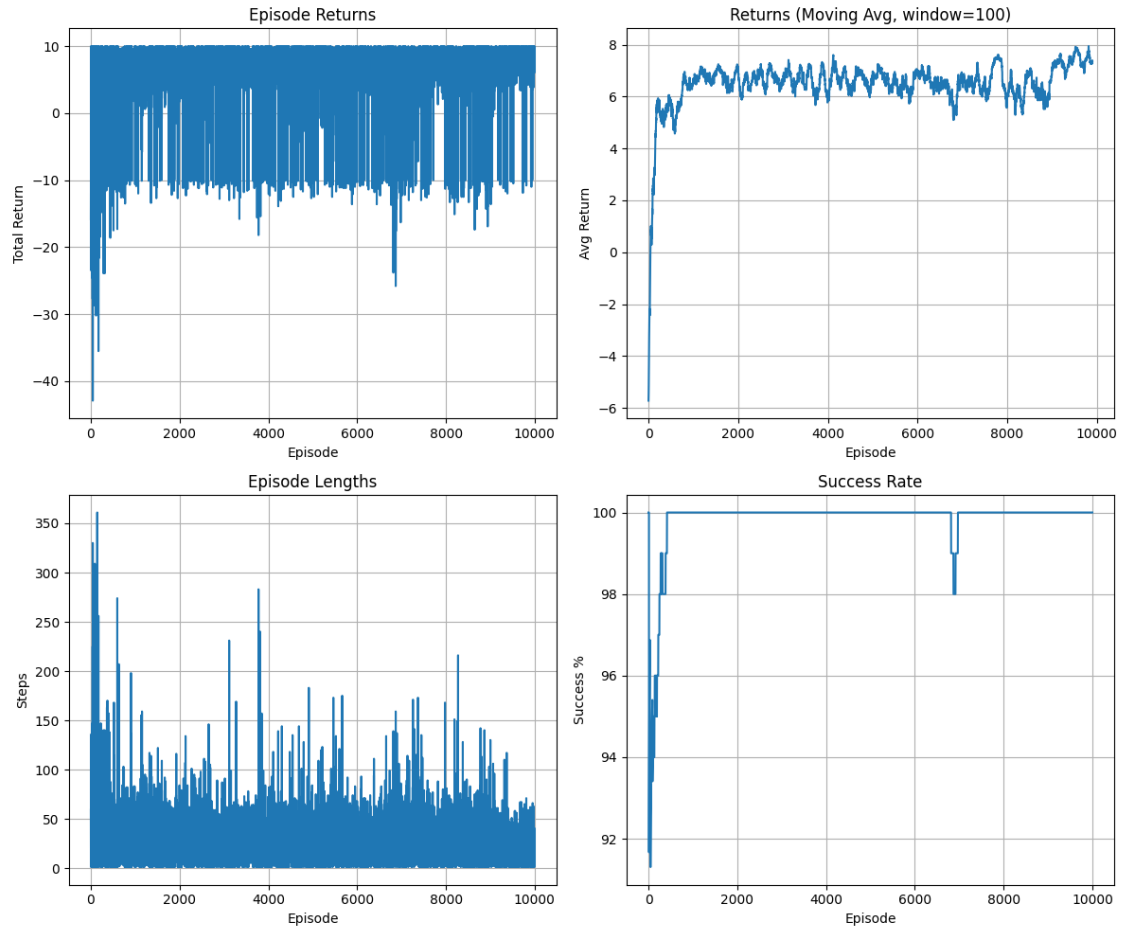


Figura 1: Q-learning: retornos, media móvil (100), longitudes y éxito.

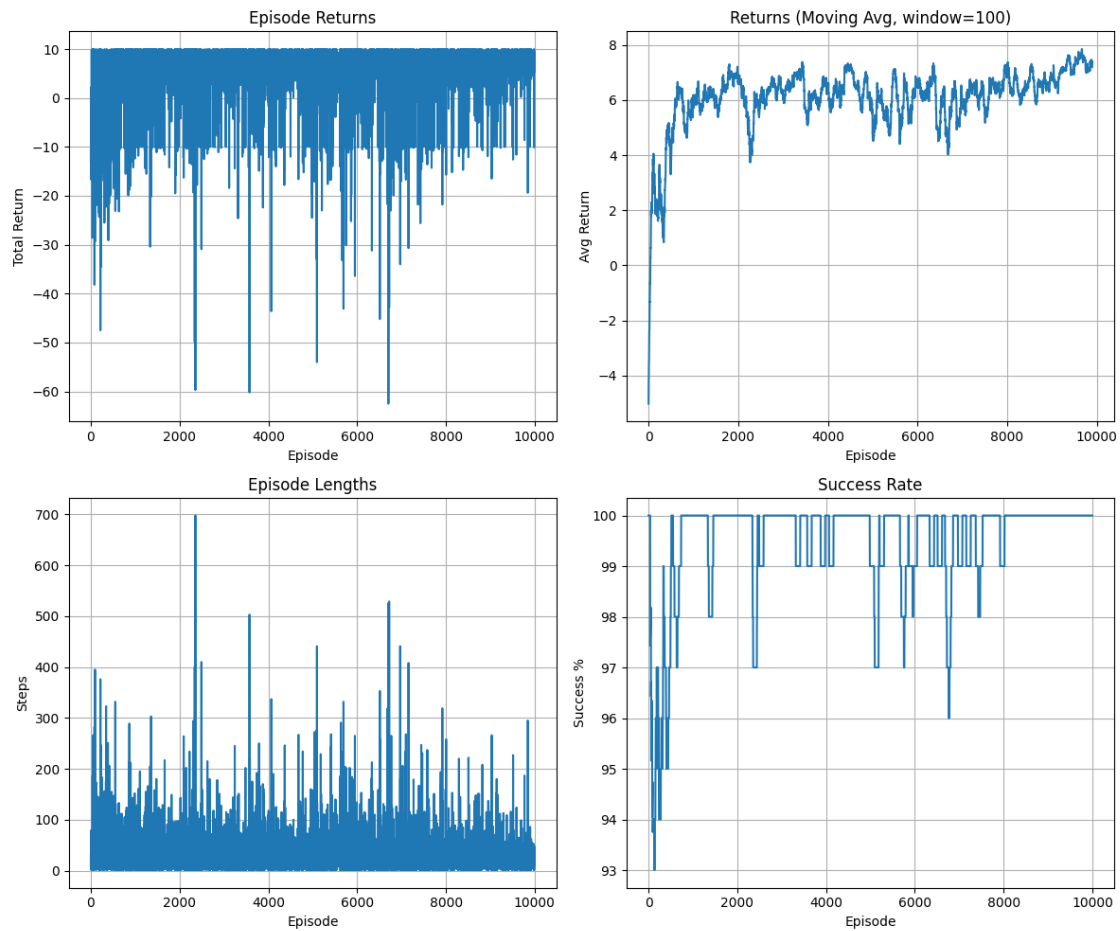


Figura 2: SARSA: retornos, media móvil (100), longitudes y éxito.

En los resultados que obtuvimos, ambos alcanzan tasas de éxito muy altas. Q-learning tiende a converger algo más rápido en la media móvil y muestra longitudes medianas ligeramente menores en los últimos episodios; SARSA exhibe mayor variabilidad episódica pero una media móvil muy estable al final. Estas diferencias son coherentes con el objetivo temporal *off-policy* de Q-learning frente a la naturaleza *on-policy* de SARSA.

2. Operación en almacén